

SIMATS ENGINEERING

ASSESSMENT TOOL 1 (CO2)

Integrated Experiments

NAME: P.Sameera Banu

REG NO: 192372172

COURSE NAME: Internet Programming

COURSE CODE: CSA4308

Experiment 1: Responsive Layout Design for Retail Web Applications

(a) Analysis of Lack of Responsiveness in Existing Layouts

When a traditional retail layout is loaded on mobile and tablet screens, several visual defects—such as horizontal scrollbars, clipped text, overlapping containers, and awkward gaps—typically occur. The core technical reasons include:

- **Fixed Pixel Dimensions (px):** Hardcoding structural container widths (e.g., width: 1200px) forces elements to stay rigid regardless of screen width. When rendered on smaller displays (e.g., 375px mobile screen), the layout overflows, creating horizontal scrolling.
- **Missing Viewport Configuration:** Omission of the standard `<meta name="viewport" content="width=device-width, initial-scale=1.0">` tag causes mobile devices to fall back to desktop layout rendering (~980px), forcing the entire webpage to shrink down to a tiny, unreadable scale.
- **Static Multi-Column Grids:** Without flex wrapping or media queries, multi-column sidebars and product grids fail to collapse into single-column vertical stacks on narrow viewports.
- **Unconstrained Media Assets:** Images lacking dynamic styling rules (max-width: 100%) expand beyond their parent card boundaries, pushing adjacent content out of alignment.
- **Lack of Spacing Scale:** Hardcoded padding and margins fail to shrink on smaller screens, leading to awkward gaps or cramped content blocks.

(b, c, & d) HTML & CSS Implementation

The following complete codebase builds a modern, responsive retail interface. Save the HTML code as `index.html` and the CSS code as `styles.css` in the same directory.

Code:

HTML Code:

```
C:\Users\Dell\Documents\IP Assessment tools\success.html nav.site-nav > ul
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Aura Store | Premium Retail Experience</title>
7    <link rel="stylesheet" href="q1.css">
8  </head>
9  <body>
10
11    <div class="announcement-bar">
12      ⚡ Free Express Shipping on orders over $100 | Use code <strong>AURA20</strong>
13    </div>
14
15    <header class="site-header">
16      <div class="logo">AURA<span>.</span></div>
17      <div class="search-bar">
18        <input type="text" placeholder="Search premium items...">
19      </div>
20      <div class="header-actions">
21        <a href="#" class="btn-secondary">Sign In</a>
22        <a href="#" class="btn-primary">Cart (2)</a>
23      </div>
24    </header>
25
26    <nav class="site-nav">
27      <ul>
28        <li><a href="#" class="active">Home</a></li>
29        <li><a href="#">New Arrivals</a></li>
30        <li><a href="#">Electronics</a></li>
31        <li><a href="#">Lifestyle</a></li>
32        <li><a href="#">Deals</a></li>

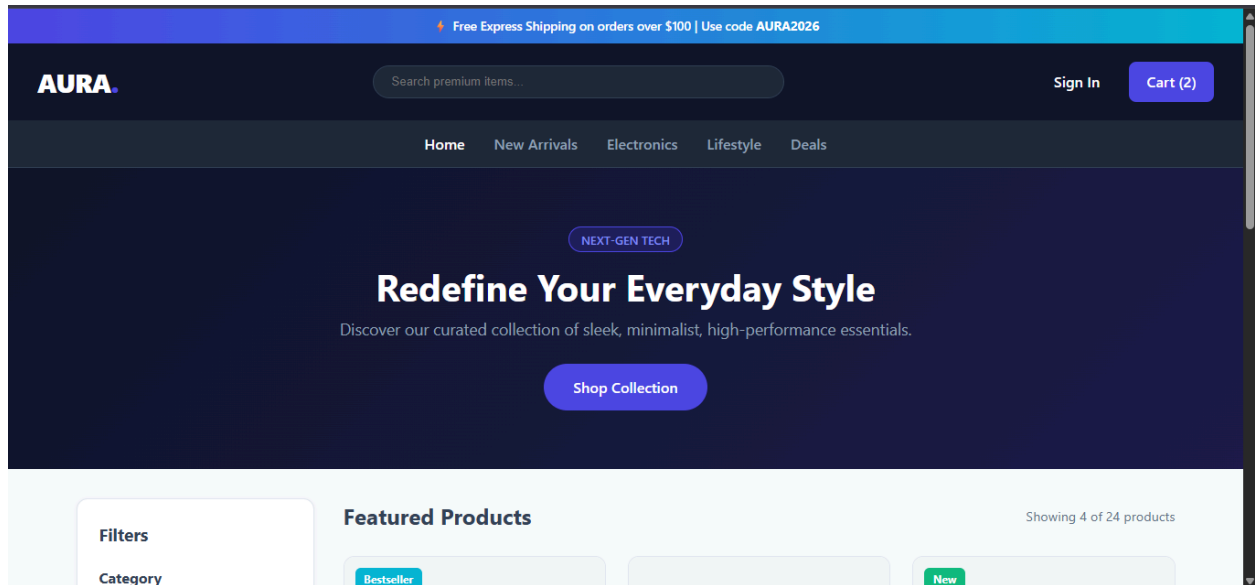
```

CSS Code:

```
CO1-AT3 > CO2-AT1 > # q1.css > {} @media (max-width: 600px)
1  /* Modern CSS Reset & Variable Theme */
2  √ :root {
3      --primary: #4f46e5;
4      --primary-hover: #4338ca;
5      --dark-bg: #0f172a;
6      --nav-bg: #1e293b;
7      --text-main: #334155;
8      --text-muted: #64748b;
9      --bg-light: #f8fafc;
10     --card-bg: #ffffff;
11     --border-color: #e2e8f0;
12     --shadow-sm: 0 1px 3px rgba(0,0,0,0.05);
13     --shadow-md: 0 10px 15px -3px rgba(0,0,0,0.08);
14     --shadow-lg: 0 20px 25px -5px rgba(0,0,0,0.1);
15     --radius-md: 12px;
16     --radius-sm: 8px;
17 }
18
19 √ * {
20     box-sizing: border-box;
21     margin: 0;
22     padding: 0;
23 }
24
25 √ body {
26     font-family: 'Inter', system-ui, -apple-system, sans-serif;
27     color: var(--text-main);
28     background-color: var(--bg-light);
29     line-height: 1.6;
30     display: flex;
31     flex-direction: column;
32     min-height: 100vh;
```

Ln 404, Col 2 Spaces: 4 UTF-8 C

Output:



(e) Evaluation of Effectiveness & Further Suggested Improvements

Evaluation of Solution

- **Fluid Grid Adaptability:** Using `repeat(auto-fit, minmax(230px, 1fr))` automatically calculates screen real estate and wraps product cards smoothly across any screen resolution without broken boundaries.
- **Layout Structure Separation:** Using **CSS Grid** for the macro layout (sidebar + main content container) combined with **Flexbox** for micro components (header items, nav link alignment, pricing rows) ensures structural stability.
- **Modern Visual Hierarchy:** Subtle hover states (`transform: translateY(-6px)`), CSS variable usage (`:root`), rounded card borders (12px), drop shadows, and dark-themed navigation sections deliver an intuitive, clean interface.

Further Suggested Improvements

1. **Dynamic Mobile Drawer:** Implement a JavaScript-powered hamburger menu on screens under 600px to collapse navigation links into an accessible slide-out drawer, preserving screen real estate.
2. **Fluid Typography Scaling:** Implement CSS `clamp()` functions (e.g., `font-size: clamp(1.8rem, 4vw, 2.5rem);`) for dynamic heading scaling across screens without reliance on media query breakpoints.

3. **Adaptive Image Delivery:** Incorporate HTML5 `<picture>` elements paired with `srcset` attributes to deliver lightweight, WebP-formatted image formats tailored to mobile display densities, boosting loading speeds on mobile data.

Experiment 2: Form Validation using JavaScript

(a) Analysis of Issues in Existing Form Validation

When a registration form accepts invalid email addresses and phone numbers, it leads to corrupted user databases, failed notification delivery, and security vulnerabilities. Key flaws in naive validation mechanisms include:

1. **Generic Input Types:** Using standard `<input type="text">` instead of semantic `<input type="email">` or `<input type="tel">` fails to trigger mobile browser soft keymaps (like @ key or numeric keypads).
2. **Submit-Only Feedback:** Validating data only upon submit button clicks forces users into a tedious cycle of submitting, waiting, reading errors, and re-submitting.
3. **Missing Pattern Matching:** Accepting raw text strings without structural rules allows syntactically broken inputs (e.g., `user@com` or `abc1234`).
4. **Poor Error Messaging:** Absence of dedicated DOM elements (`<span>` error targets) leaves users guessing what format is expected.
5. **Over-reliance on Client Validation:** Relying solely on client-side checks without backend validation exposes systems to script bypass attacks using tools like cURL or Postman.

b) Input Fields Design with HTML5 Attributes

Using semantic types, placeholder guides, autocomplete, and required attributes ensures accessibility and browser-level initial checks.

```
<div class="form-group" id="emailGroup">  
  <label for="email">Email Address</label>  
  <input  
    type="email"  
    id="email"  
    name="email"  
    placeholder="e.g. alex@example.com"  
    required  
    autocomplete="email"  
  >  
  <span class="error-message" id="emailError"></span>  
</div>
```

```
<div class="form-group" id="phoneGroup">  
  <label for="phone">Phone Number</label>  
  <input  
    type="tel"  
    id="phone"  
    name="phone"  
    placeholder="e.g. +1 1234567890 or 9876543210"  
    required  
    autocomplete="tel"  
  >
```


</div>

(c) & (d) JavaScript Implementation with RegEx & Real-Time Error Display

Save the following code in index.html, styles.css, and script.js.

Code:

HTML:

```
CO1-AT3 > CO2-AT1 > AT1CO2EXP2 > index.html > html > head > link
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Real-Time Form Validation</title>
7    <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10
11    <div class="form-card">
12      <h2>Account Registration</h2>
13      <p class="subtitle">Please enter your contact details below.</p>
14
15      <form id="registrationForm" novalidate>
16
17        <div class="form-group" id="emailGroup">
18          <label for="email">Email Address</label>
19          <input
20            type="email"
21            id="email"
22            name="email"
23            placeholder="e.g. alex@example.com"
24            required
25            autocomplete="email"
26          >
27          <span class="error-message" id="emailError"></span>
28        </div>
29
30        <div class="form-group" id="phoneGroup">
31          <label for="phone">Phone Number</label>
32          <input
```

CSS:

```
CO1-AT3 > CO2-AT1 > AT1CO2EXP2 > # style.css > .success-message
1  * {
2    box-sizing: border-box;
3    margin: 0;
4    padding: 0;
5  }
6
7  body {
8    font-family: 'Inter', system-ui, -apple-system, sans-serif;
9    background-color: #f1f5f9;
10   color: #334155;
11   display: flex;
12   justify-content: center;
13   align-items: center;
14   min-height: 100vh;
15   padding: 1rem;
16 }
17
18 .form-card {
19   background: #ffffff;
20   padding: 2.5rem;
21   border-radius: 16px;
22   box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.1);
23   width: 100%;
24   max-width: 440px;
25 }
26
27 .form-card h2 {
28   font-size: 1.6rem;
29   color: #0f172a;
30 }
31
32 .subtitle {
```

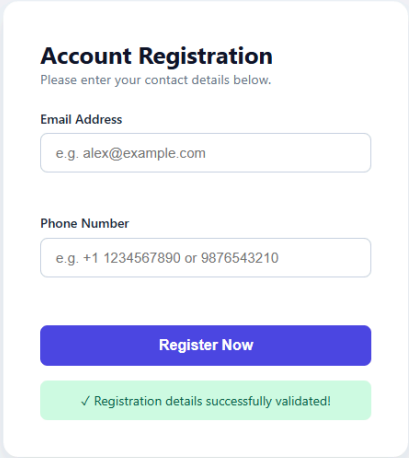
Ln 111, Col 2 Spaces: 4

Java Script:

```
CO1-AT3 > CO2-AT1 > AT1CO2EXP2 window.document returns a reference to the document contained in the win
1  document.addEventListener('submit', (e) => {
2    const form = document.getElementById('form');
3    const emailInput = document.getElementById('email');
4    const phoneInput = document.getElementById('phone');
5
6    const emailGroup = document.getElementById('emailGroup');
7    const phoneGroup = document.getElementById('phoneGroup');
8
9    const emailError = document.getElementById('emailError');
10   const phoneError = document.getElementById('phoneError');
11   const formSuccess = document.getElementById('formSuccess');
12
13   // Regex Patterns
14   // Email: Standard alphanumeric + domain structure
15   const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
16
17   // Phone: Supports 10-digit formats and international +country codes
18   const phoneRegex = /^(+?\d{1,3}[-.\s]?)?(+?\d{3}\s?)?[-.\s]?\d{3}[-.\s]?\d{4}$/;
19
20   // Email Validation Logic
21   function validateEmail() {
22     const value = emailInput.value.trim();
23     if (value === '') {
24       showError(emailGroup, emailError, 'Email address is required.');
```

Ln 88, Col 4 Spaces: 4 UTF

Output:



Account Registration
Please enter your contact details below.

Email Address
e.g. alex@example.com

Phone Number
e.g. +1 1234567890 or 9876543210

Register Now

✓ Registration details successfully validated!

(e) Evaluation & System Enhancements

System Evaluation

- **Immediate Real-Time Feedback:** Listening to input and blur events prevents submission friction by guiding users right while typing.
- **Precise Regular Expression Rules:** Standard RFC-compliant email checks and flexible phone format matching reduce bad data entry.
- **Accessibility & Visual Indicators:** Contextual CSS class toggling (.valid / .invalid) visually signals state changes through color tints and dedicated text nodes.

Enhancements for UX & Security

1. **Server-Side Validation:** Always re-validate all inputs on the backend server before database entry to prevent malicious script injection or API bypasses.
2. **Input Debouncing:** Implement a ~300ms debounce function on real-time event listeners to optimize execution efficiency during rapid key presses.
3. **2-Factor Verification (OTP):** Combine structural RegEx validation with an SMS/Email OTP (One-Time Password) service to verify input ownership.

4. **ARIA Screen Reader Support:** Dynamically toggle `aria-invalid="true"` and `aria-describedby` attributes for improved accessibility compliance.

Experiment 3: CSS Layout Debugging & Responsiveness

(a) Analysis of CSS Properties Causing Layout Overlapping

Layout overlapping occurs when elements breach their intended DOM boundaries during browser resizing. The root CSS causes include:

1. **Misuse of position: absolute & fixed:** Absolute positioning removes elements completely from the normal document flow. Sibling elements render as if the absolute element does not exist, causing collisions when screen size changes.
2. **Hardcoded Pixel Dimensions (px):** Setting fixed width or height values (e.g., width: 900px, height: 400px) prevents containers from flexing or wrapping as the browser viewport shrinks.
3. **Default Box-Sizing Model (content-box):** Under the default content-box behavior, padding and borders are added *on top* of the declared width (Total Width = Specified Width + Padding + Border). A container set to width: 100% with padding: 20px will exceed 100% viewport width, breaking multi-column layouts and overlapping adjacent components.

4. **Uncleared Floating Elements:** Legacy float: left layouts without proper clearfix containers collapse parent container heights to zero, letting lower section content slip underneath the floating elements.

(b) Role of the CSS Box Model in Layout Design

The CSS Box Model is the foundation of element rendering and layout calculation. Every rendered HTML element is treated as a rectangular box comprising four concentric layers:

- **Content:** The innermost area where text, images, or child elements reside.
- **Padding:** Clear area surrounding the content, inside the border boundary.
- **Border:** The edge surrounding the padding and content layers.
- **Margin:** Clear area outside the border, separating the element from neighboring elements.

Box Model Sizing Comparison

- **box-sizing: content-box (Default):**

$$\text{Rendered Width} = \text{Width} + \text{Padding Left/Right} + \text{Border Left/Right}$$

Issue: Causes width calculation errors during window resizing.

- **box-sizing: border-box (Modern Standard):**

$$\text{Rendered Width} = \text{Explicitly Set Width}$$

Advantage: Padding and borders are absorbed *inside* the width, eliminating unexpected container overflows and layout breaks.

(c) & (d) Debugging Strategy & Modified CSS Code

To resolve layout collisions:

- Global box-sizing: border-box is enforced.

- Absolute positioning is replaced with **Flexbox** for 1D alignments and **CSS Grid** for 2D multi-column layouts.
- Flex limits (flex: 1 1 240px) and auto-fit column definitions are added to ensure fluid wrapping.

Code:

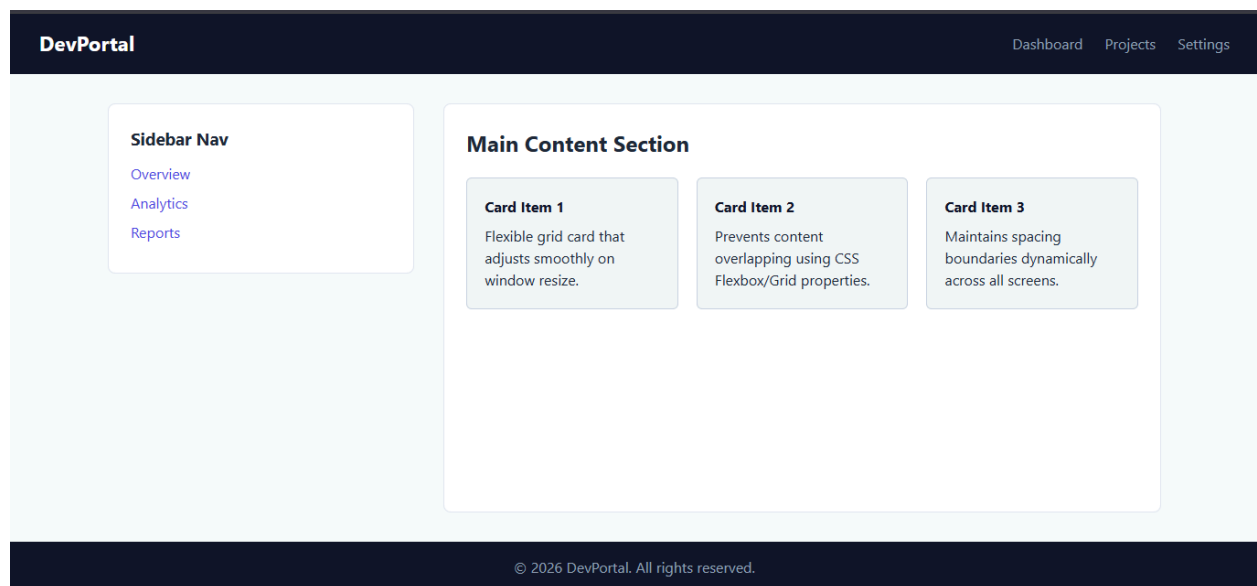
HTML:

```
CO1-AT3 > CO2-AT1 > AT1CO2EXP2 > AT1CO2EXP3 > index.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Layout Debugging & Alignment</title>
7    <link rel="stylesheet" href="styles.css">
8  </head>
9  <body>
10
11    <header class="header">
12      <div class="logo">DevPortal</div>
13      <nav class="nav">
14        <a href="#">Dashboard</a>
15        <a href="#">Projects</a>
16        <a href="#">Settings</a>
17      </nav>
18    </header>
19
20    <div class="main-container">
21      <aside class="sidebar">
22        <h3>Sidebar Nav</h3>
23        <ul>
24          <li><a href="#">Overview</a></li>
25          <li><a href="#">Analytics</a></li>
26          <li><a href="#">Reports</a></li>
27        </ul>
28      </aside>
29
30      <main class="content-area">
31        <h2>Main Content Section</h2>
32        <div class="card-grid">
```

CSS:

```
CO1-AT3 > CO2-AT1 > AT1CO2EXP2 > AT1CO2EXP3 > # styles.css > .footer
1  /* =====
2  GLOBAL BOX-MODEL RESET & PREVENT OVERFLOW (b & d)
3  ===== */
4  *, *::before, *::after {
5    box-sizing: border-box; /* Includes padding & border in explicit width
6    margin: 0;
7    padding: 0;
8  }
9
10 body {
11   font-family: 'Inter', system-ui, -apple-system, sans-serif;
12   line-height: 1.5;
13   color: #1e293b;
14   background-color: #f8fafc;
15   display: flex;
16   flex-direction: column;
17   min-height: 100vh;
18 }
19
20 /* =====
21 HEADER: FLEXBOX ALIGNMENT (NO OVERLAPPING)
22 ===== */
23 .header {
24   background-color: #0f172a;
25   color: #ffffff;
26   padding: 1rem 2rem;
27   display: flex;
28   justify-content: space-between;
29   align-items: center;
30   flex-wrap: wrap; /* Allows header links to wrap smoothly on tiny viewports */
31   gap: 1rem;
32 }
```

Output:



(e) Layout Evaluation & Best Practices

Evaluation of Corrected Layout

- **Fluid Flow Adaptability:** Replacing static width: px with Flexbox properties (flex: 1 1 240px) and CSS Grid (repeat(auto-fit, minmax(220px, 1fr))) allows content to reflow naturally without overlapping during window resize.
- **Controlled Document Flow:** Eliminating position: absolute on core page containers ensures every element respects sibling boundaries.
- **Predictable Sizing Calculations:** Applying box-sizing: border-box globally ensures container padding never breaks maximum width limits.

Best Practices for Layout Consistency

1. **Global Box-Model Reset:** Always initialize stylesheets with *, *::before, *::after { box-sizing: border-box; }.
2. **Use Relative Units:** Prefer percentages (%), root ems (rem), fractional units (fr), and viewport units (vw/vh) over hardcoded pixel dimensions.
3. **Restrict Absolute Positioning:** Reserve position: absolute exclusively for non-structural UI overlays like popups, tooltips, dropdown menus, or icon badges.
4. **Leverage Modern CSS Layout Engines:** Combine CSS Grid for 2D structural layouts (main layout areas) with Flexbox for 1D component positioning (headers, navbars, button groups).
5. **Responsive DevTools Testing:** Regularly inspect layouts across standard viewport breakpoints (375px, 768px, 1024px, 1440px) to identify alignment bottlenecks early.